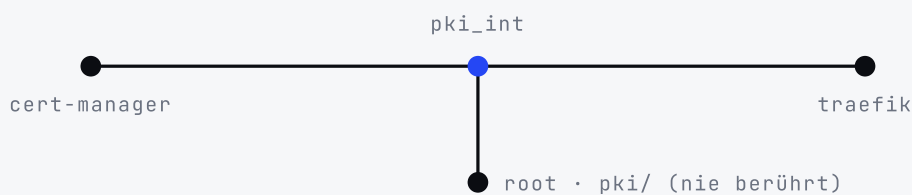


INTERNE PKI MIT OPENBAO

Eine interne CA, deren Schlüssel den Tresor nie verlässt:
Root und Intermediate in OpenBao, Zertifikate per cert-
manager, Erneuerung und Widerruf.



PKI mit OpenBao und cert-manager

Eine interne CA, deren Schlüssel den Tresor nie verlässt: Root und Intermediate in OpenBao, Zertifikate per cert-manager für Ingress und Service-zu-Service, Erneuerung und Widerruf

Thomas Zachmann

17. September 2026

Inhalt

Worum es geht

- Warum von Hand, und warum ohne KI

- Lizenz und Haftung

- Für wen

- Die Bausteine

- Die Architektur in einem Bild

Teil I – Der Ausgangspunkt

Teil II – Root und Intermediate in OpenBao

- Root-CA

- Intermediate-CA

- Die Rolle: was signiert wird

- Policy und Auth-Rolle für cert-manager

Teil III – cert-manager anbinden

- ServiceAccount, RBAC, ClusterIssuer

- Das erste Zertifikat

- Die Kette prüfen

Teil IV – Was abgelehnt wird

Teil V – Ingress mit ingress-shim

Teil VI – Erneuerung und Widerruf

- Erneuerung

- Widerruf

Teil VII – Betrieb

- Das Root verteilen

- Die alte CA ablösen

- Root-Rotation, in fünf Jahren

- Was in OpenTofu gehört

- Was noch offen ist

Anhang A – Alle Kommandos

Anhang B – Glossar

Über den Autor

Worum es geht

Jeder Cluster braucht irgendwann eine eigene CA. Für Ingress-Hosts, die nicht öffentlich sind, für TLS zwischen Diensten, für Webhooks, für alles, was ein Zertifikat will und keinen Let's-Encrypt-Weg hat. Der übliche Weg ist cert-manager mit einem selbstsignierten Root: Zwei Manifeste, fünf Minuten, fertig – und der private Schlüssel der CA liegt als Kubernetes-Secret im Namespace cert-manager. Jeder mit `get secrets` dort kann Zertifikate für jeden Hostnamen im Cluster ausstellen, und niemand erfährt es.

Dieser Leitfaden verlegt die CA nach OpenBao. Root- und Intermediate-CA werden dort erzeugt, der Schlüssel verlässt OpenBao nie. cert-manager erzeugt weiterhin die privaten Schlüssel der Zertifikate im Cluster, schickt aber nur noch den CSR an OpenBao – über eine Rolle, die festlegt, welche Hostnamen, welche Schlüsseltypen und welche Laufzeiten erlaubt sind. Jede Ausstellung ist in OpenBao nachvollziehbar, jeder Widerruf landet auf der CRL.

Es ist der fünfte Teil einer Reihe über OpenBao im Lab. N° 1 baut den OpenBao im Cluster, N° 2 und N° 4 lassen den External Secrets Operator Secrets daraus beziehen. Dieser Leitfaden ist unabhängig davon lesbar, setzt aber den OpenBao aus N° 1 mit aktivierter Kubernetes-Auth voraus.

Alles wurde am Cluster durchgeführt. Die zwei Fehler, die auftraten, sind drin – einer davon war nicht der erwartete.

Warum von Hand, und warum ohne KI

Eine CA-Hierarchie ist in zwanzig Zeilen HCL beschrieben, und ein Sprachmodell schreibt sie fehlerfrei. Was es nicht mitliefert: warum der Intermediate-CSR OpenBao verlässt, das Root aber nie; warum `set-signed` plötzlich zwei Issuer erzeugt; warum ein Zertifikat mit `subject=` leer trotzdem gültig ist; und warum die erste Ablehnung nicht am Hostnamen lag, sondern am Schlüsseltyp.

Werkzeuge wie [nyrvex](#), das die Konfiguration von Secret Store und Identity Provider einer AI-Plattform generiert, nehmen einem diese Schritte später ab. Man sollte sie einmal selbst gegangen sein, um beurteilen zu können, was generiert wurde.

Der Maßstab: Wer diesen Leitfaden durchgearbeitet hat, kann auf einem leeren Blatt zeichnen, welcher Schlüssel wo entsteht, welches Artefakt zwischen Cluster und OpenBao wandert, und an welcher Stelle eine Rolle entscheidet, ob ein Zertifikat ausgestellt wird.

Lizenz und Haftung

Dieser Leitfaden steht unter CC BY 4.0: Er darf kopiert, weitergegeben und bearbeitet werden, auch kommerziell, solange der Autor genannt wird. Er wird ohne Gewähr bereitgestellt. Alles darin wurde in einer Entwicklungsumgebung durchgeführt; wer es in einer anderen Umgebung nachvollzieht, tut das auf eigene Verantwortung.

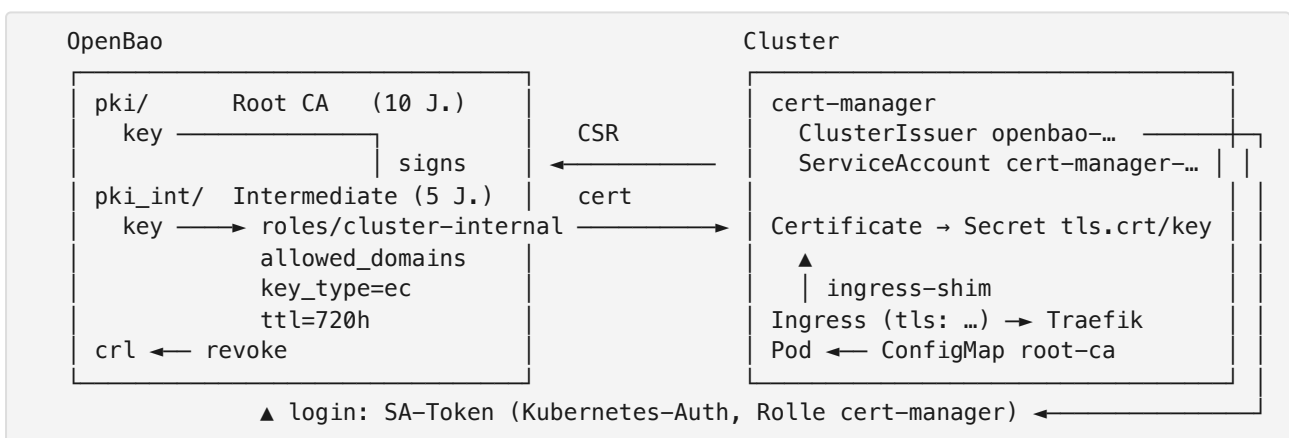
Für wen

Für Leserinnen und Leser, die cert-manager schon einmal benutzt haben und wissen, was ein CSR ist. OpenBao-Grundlagen – Mounts, Rollen, Policies, Kubernetes-Auth – werden vorausgesetzt; N° 1 dieser Reihe oder mein Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice) liefern sie.

Die Bausteine

Baustein	Version	Rolle
OpenBao im Cluster (N° 1)	2.6.2	PKI-Engines pki/ und pki_int/, Kubernetes-Auth
cert-manager	v1.20.2	Issuer vom Typ vault, Certificate, ingress-shim
Traefik	RKE2-Chart 40.1	Ingress mit TLS aus dem cert-manager-Secret
Miniflux (N° 2)	2.3.3	der Dienst, der ein Zertifikat bekommt

Die Architektur in einem Bild



Drei Dinge, die man aus dem Bild mitnehmen sollte:

1. **Zwei Schlüssel, zwei Orte.** Der CA-Schlüssel entsteht in OpenBao und bleibt dort. Der Zertifikatsschlüssel entsteht im Cluster (cert-manager) und bleibt dort. Nur der CSR und das signierte Zertifikat wandern.
2. **Die Rolle ist die Policy der PKI.** Was OpenBao signiert, entscheidet nicht cert-manager, sondern pki_int/roles/cluster-internal: erlaubte Domains, Schlüsseltyp, Laufzeit. cert-manager kann nur beantragen.
3. **Root und Intermediate sind getrennte Mounts.** Das Root signiert genau einmal – den Intermediate – und wird danach nicht mehr angefasst. Wird der Intermediate kompromittiert, widerruft man ihn und stellt einen neuen aus; das Root bleibt vertrauenswürdig.

Teil I – Der Ausgangspunkt

So sah der Cluster vor diesem Leitfaden aus:

```
$ kubectl get clusterissuer
NAME                READY   AGE
homelab-ca          True    3d22h
selfsigned-bootstrap True    3d22h

$ kubectl get clusterissuer homelab-ca -o jsonpath='{.spec}'
{"ca":{"secretName":"homelab-ca-key-pair"}}

$ kubectl get certificate -n cert-manager homelab-ca -o jsonpath='{.spec.isCA} {.spec.duration}'
true 87600h
```

Ein selbstsigniertes Root, zehn Jahre gültig, Schlüssel in cert-manager/homelab-ca-key-pair. Zwei Dienste beziehen Zertifikate daraus. Das funktioniert, und es ist der Stand, den die meisten Cluster haben.

Was daran fehlt: Der Schlüssel liegt in etcd, lesbar für jeden mit Secret-Rechten im Namespace. Es gibt keinen Intermediate – jedes Zertifikat hängt direkt am Root. Es gibt keine Aufzeichnung, wer wann welches Zertifikat ausgestellt hat, außer den CertificateRequest-Objekten, die irgendwann aufgeräumt werden. Und es gibt keinen Widerruf: cert-manager kennt keine CRL.

Dieser Leitfaden lässt die alte CA stehen – ein Umzug laufender Dienste ist ein eigenes Thema (Teil VII) – und baut die neue daneben.

Teil II – Root und Intermediate in OpenBao

Root-CA

```
/ $ bao secrets enable -path=pki -max-lease-ttl=87600h pki
/ $ bao write pki/root/generate/internal \
    common_name="Homelab Root CA" issuer_name=root-2026 \
    key_type=ec key_bits=256 ttl=87600h
```

generate/internal heißt: Der Schlüssel wird in OpenBao erzeugt und **nicht zurückgegeben**. Die Antwort enthält das Zertifikat, die Issuer-ID und die Key-ID, aber keinen privaten Schlüssel. (generate/exported gäbe ihn zurück – für Backup-Szenarien, die man dann auch wirklich braucht.)

EC P-256 statt RSA: kleinere Schlüssel, schnellere Signaturen, von allem Modernen unterstützt. ttl=87600h sind zehn Jahre – max-lease-ttl des Mounts muss mindestens so groß sein, sonst kappt OpenBao stillschweigend.

```
/ $ bao write pki/config/urls \
    issuing_certificates="http://openbao.openbao.svc:8200/v1/pki/ca" \
    crl_distribution_points="http://openbao.openbao.svc:8200/v1/pki/crl"
```

Die URLs landen als AIA und CRL-Distribution-Point in jedem ausgestellten Zertifikat. Ohne sie warnt OpenBao bei jedem Signieren, und Clients, die Widerrufe prüfen wollen, finden die CRL nicht.

Intermediate-CA

Ein zweiter Mount, eigener Schlüssel, halbe Laufzeit:

```
/ $ bao secrets enable -path=pki_int -max-lease-ttl=43800h pki
/ $ bao write -field=csr pki_int/intermediate/generate/internal \
    common_name="Homelab Intermediate CA" key_type=ec key_bits=256 > /tmp/pki_int.csr
```

Das ist der einzige Moment, in dem etwas aus dem Intermediate-Mount herauskommt – ein **CSR**, kein Schlüssel. Er geht zum Root:

```
/ $ bao write -field=certificate pki/root/sign-intermediate \
    csr=@/tmp/pki_int.csr format=pem_bundle ttl=43800h issuer_ref=root-2026 > /tmp/pki_int.pem
/ $ bao write pki_int/intermediate/set-signed certificate=@/tmp/pki_int.pem
mapping    map[0f271d38-...: 84eb727c-...:31c9adfe-...]
```

Das mapping verrät etwas, das man wissen muss: set-signed hat **zwei** Issuer angelegt. 84eb727c ist der Intermediate mit Key 31c9adfe; 0f271d38 ist eine Kopie des Root-Zertifikats **ohne** Key, importiert, damit der Mount die Kette vollständig ausliefern kann. Der beim generate vergebene issuer_name ist dabei verloren gegangen:

```
/ $ bao read -field=issuer_name pki_int/issuer/84eb727c-...  
(leer)  
/ $ bao write pki_int/issuer/84eb727c-... issuer_name=int-2026  
/ $ bao write pki_int/issuer/0f271d38-... issuer_name=root-2026-chain
```

Namen sind nicht nur Kosmetik: issuer_ref=int-2026 in einer Rolle überlebt eine Rotation, eine UUID nicht. Der Default-Issuer des Mounts ist bereits der richtige (bao read pki_int/config/issuers).

```
/ $ bao write pki_int/config/urls \  
    issuing_certificates="http://openbao.openbao.svc:8200/v1/pki_int/ca" \  
    crt_distribution_points="http://openbao.openbao.svc:8200/v1/pki_int/crl"  
/ $ bao read -field=certificate pki_int/cert/ca | openssl x509 -noout -subject -issuer -dates  
subject=CN=Homelab Intermediate CA  
issuer=CN=Homelab Root CA  
notBefore=Sep 17 11:13:42 2026 GMT  
notAfter=Sep 16 11:14:12 2031 GMT
```

Die Rolle: was signiert wird

```
/ $ bao write pki_int/roles/cluster-internal \  
    allowed_domains="svc.cluster.local,demo.svc,example.internal" \  
    allow_subdomains=true allow_bare_domains=false allow_glob_domains=false \  
    allow_ip_sans=false enforce_hostnames=true \  
    key_type=ec key_bits=256 \  
    ttl=720h max_ttl=2160h \  
    server_flag=true client_flag=false require_cn=false
```

Jedes Feld ist eine Entscheidung:

Feld	Wert	Warum
allowed_domains + allow_subdomains	Cluster-DNS und die interne Zone	alles andere wird abgelehnt – auch example.com
allow_bare_domains=false	–	svc.cluster.local selbst bekommt kein Zertifikat, nur Subdomains
allow_ip_sans=false	–	keine IP-Zertifikate; Dienste haben Namen
key_type=ec	P-256	die Rolle erzwingt den Typ, cert-manager muss ihn liefern (Teil IV)
ttl=720h, max_ttl=2160h	30 / 90 Tage	kurz genug, dass Erneuerung geübt wird
server_flag, client_flag	nur Server	ein Zertifikat für Service-zu-Service- Client -Auth bräuchte eine zweite Rolle
require_cn=false	–	moderne Clients prüfen SANs, nicht CN; cert-manager schickt oft keinen

Policy und Auth-Rolle für cert-manager

```
/ $ bao policy write cert-manager - <<'EOF'
path "pki_int/sign/cluster-internal" { capabilities = ["create", "update"] }
EOF
/ $ bao write auth/kubernetes/role/cert-manager \
    bound_service_account_names=cert-manager-openbao \
    bound_service_account_namespaces=cert-manager \
    token_policies=cert-manager token_ttl=10m
```

Eine Zeile Policy. cert-manager darf genau einen Pfad aufrufen: sign mit dieser Rolle. Nicht issue (das würde OpenBao den Schlüssel erzeugen lassen), nicht pki_int/*, nichts am Root.

Das vollständige OpenBao-Setup steht in openbao/pki-setup.sh.

Teil III – cert-manager anbinden

ServiceAccount, RBAC, ClusterIssuer

cert-manager meldet sich nicht mit seiner eigenen Identität bei OpenBao an, sondern mit dem Token eines eigens dafür angelegten ServiceAccounts. Dafür braucht es das Recht, Tokens für diesen Account zu erzeugen:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cert-manager-openbao
  namespace: cert-manager
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cert-manager-openbao-token
  namespace: cert-manager
rules:
  - apiGroups: [""]
    resources: ["serviceaccounts/token"]
    resourceNames: ["cert-manager-openbao"]
    verbs: ["create"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cert-manager-openbao-token
  namespace: cert-manager
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: cert-manager-openbao-token
subjects:
  - kind: ServiceAccount
    name: cert-manager
    namespace: cert-manager
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: openbao-cluster-internal
spec:
  vault:
    server: http://openbao.openbao.svc:8200
    path: pki_int/sign/cluster-internal
    auth:
      kubernetes:
        mountPath: /v1/auth/kubernetes
        role: cert-manager
        serviceAccountRef:
          name: cert-manager-openbao
```

Der ServiceAccount liegt im Namespace cert-manager, weil ein ClusterIssuer seinen serviceAccountRef dort auflöst. Bei einem namespace-gebundenen Issuer läge er im Namespace des Issuers – dann kann man pro Namespace eine eigene OpenBao-Rolle und eigene allowed_domains vergeben. Für einen Lab-Cluster reicht der ClusterIssuer.

```
$ kubectl apply -f k8s/clusterissuer.yaml
$ kubectl get clusterissuer openbao-cluster-internal
NAME                                READY   AGE
openbao-cluster-internal            True    8s
$ kubectl get clusterissuer openbao-cluster-internal -o jsonpath='{.status.conditions[0].message}'
Vault verified
```

Beim ersten Versuch Ready. Der Issuer-Typ heißt vault, weil OpenBao die Vault-API behalten hat; cert-manager kennt keinen Unterschied.

Das erste Zertifikat

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: miniflux-internal
  namespace: demo
spec:
  secretName: miniflux-internal-tls
  duration: 720h
  renewBefore: 240h
  privateKey:
    algorithm: ECDSA
    size: 256
    rotationPolicy: Always
  dnsNames:
  - miniflux.demo.svc
  - miniflux.demo.svc.cluster.local
  issuerRef:
    kind: ClusterIssuer
    name: openbao-cluster-internal
```

```
$ kubectl apply -f k8s/miniflux-certificate.yaml
$ kubectl get certificate,certificaterequest -n demo
NAME                                READY   SECRET                                AGE
certificate.cert-manager.io/miniflux-internal  True    miniflux-internal-tls              9s

NAME                                APPROVED  READY   ISSUER
certificaterequest.cert-manager.io/miniflux-internal-1  True      True    openbao-cluster-internal
```

Was OpenBao ausgestellt hat:

```
$ kubectl get secret -n demo miniflux-internal-tls -o jsonpath='{.data.tls\.crt}' \
| base64 -d | openssl x509 -noout -subject -issuer -dates -ext subjectAltName
subject=
issuer=CN=Homelab Intermediate CA
notBefore=Sep 17 11:15:03 2026 GMT
notAfter=Oct 17 11:15:33 2026 GMT
X509v3 Subject Alternative Name: critical
      DNS:miniflux.demo.svc, DNS:miniflux.demo.svc.cluster.local
```

subject= ist leer – kein CN. Das ist korrekt und seit Jahren Standard: Browser und TLS-Bibliotheken prüfen die SANs. `require_cn=false` in der Rolle erlaubt es, und cert-manager schickt ohne `commonName` im Manifest keinen.

`rotationPolicy: Always` sorgt dafür, dass bei jeder Erneuerung ein neuer privater Schlüssel entsteht. Der Default `Never` behält den Schlüssel über Erneuerungen hinweg – bequem, aber ein kompromittierter Schlüssel bleibt dann so lange gültig wie das Certificate-Objekt existiert.

Die Kette prüfen

Das Secret enthält drei Dinge:

```
$ kubectl get secret -n demo miniflux-internal-tls -o jsonpath='{.data}' | ...
ca.crt    615 bytes, 1 cert    ← das Root
tls.crt   1689 bytes, 2 certs  ← Leaf + Intermediate
tls.key   227 bytes
```

`tls.crt` ist die **Kette**, die ein Server ausliefert: erst das eigene Zertifikat, dann der Intermediate. `ca.crt` ist das Root, das ein Client vertrauen muss. Die Prüfung von Hand, mit dem Root direkt aus OpenBao:

```
$ bao read -field=certificate pki/cert/ca > root.pem
$ openssl verify -CAfile root.pem -untrusted intermediate.pem leaf.pem
leaf.pem: OK
```

`-untrusted` ist die richtige Stelle für den Intermediate: Er wird zur Kettenbildung benutzt, aber ihm wird nicht per se vertraut – das tut nur das Root aus `-CAfile`.

Teil IV – Was abgelehnt wird

Ein Zertifikat für `miniflux.example.com` – eine Domain, die nicht in `allowed_domains` steht. Erwartet: Ablehnung wegen des Hostnamens. Bekommen:

```
$ kubectl get certificaterequest -n demo not-allowed-1 -o jsonpath='{.status.conditions[?(@.type=="Ready")].message}'
Vault failed to sign certificate: failed to sign certificate by vault: Error making API request.

URL: POST http://openbao.openbao.svc:8200/v1/pki_int/sign/cluster-internal
Code: 400. Errors:

* role requires keys of type ec
```

Das Test-Manifest hatte keinen `privateKey`-Block. `cert-manager` nimmt dann **RSA 2048** – und die Rolle verlangt EC. OpenBao prüft den Schlüsseltyp **vor** den Hostnamen. Zwei Konsequenzen: Jedes Certificate gegen diesen Issuer braucht `privateKey.algorithm: ECDSA`, und bei Ingress-Annotationen (Teil V) muss der Algorithmus ebenfalls gesetzt werden. Oder man stellt die Rolle auf `key_type=any` – dann entscheidet der Antragsteller, was in einer Lab-PKI vertretbar ist, in einer Firmen-PKI eher nicht.

Mit ECDSA im Manifest kommt der erwartete Fehler:

```
Code: 400. Errors:

* subject alternate name miniflux.example.com not allowed by this role
```

Beides sind Ablehnungen durch OpenBao, nicht durch `cert-manager`. Der `CertificateRequest` bleibt mit `READY False` stehen, `cert-manager` versucht es mit Backoff erneut, und die Meldung steht in den Conditions. Genau so soll eine PKI sich verhalten: Die Rolle entscheidet, der Client bekommt eine klare Antwort.

Teil V – Ingress mit ingress-shim

Für Ingress-Hosts muss man kein Certificate schreiben. Eine Annotation genügt, cert-manager erzeugt das Objekt aus spec.tls:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: miniflux
  namespace: demo
  annotations:
    cert-manager.io/cluster-issuer: openbao-cluster-internal
    cert-manager.io/private-key-algorithm: ECDSA
spec:
  ingressClassName: traefik
  tls:
    - hosts:
        - miniflux.demo.example.internal
      secretName: miniflux-ingress-tls
  rules:
    - host: miniflux.demo.example.internal
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: miniflux
                port:
                  number: 8080
```

Die zweite Annotation ist die Lehre aus Teil IV. Ohne sie: RSA, Ablehnung, kein TLS am Ingress.

```
$ kubectl apply -f k8s/miniflux-service-ingress.yaml
$ kubectl get certificate -n demo
NAME                                READY    SECRET                                AGE
miniflux-ingress-tls               True     miniflux-ingress-tls                 10s
miniflux-internal                   True     miniflux-internal-tls                 90s
```

Prüfung gegen Traefik – per SNI, weil der Hostname keinen DNS-Eintrag hat:

```
$ openssl s_client -connect <worker-ip>:443 -servername miniflux.demo.example.internal -CA
file root.pem
issuer=CN=Homelab Intermediate CA
Verification: OK
Verify return code: 0 (ok)

$ curl --cacert root.pem --resolve miniflux.demo.example.internal:443:<worker-ip> \
  https://miniflux.demo.example.internal/ -o /dev/null -w '%{http_code}\n'
200
```

Traefik liefert Leaf und Intermediate aus dem Secret, der Client hat das Root, die Kette schließt. Das ist derselbe Hostname-Typ, für den der Reverse-Proxy vor dem Cluster bisher ein öffentliches Zertifikat brauchte – für rein interne Dienste reicht ab jetzt die eigene CA.

Teil VI – Erneuerung und Widerruf

Erneuerung

cert-manager erneuert automatisch, sobald `renewBefore` erreicht ist – hier zehn Tage vor Ablauf. Erzwingen lässt sich das, indem man das Secret löscht:

```
$ kubectl delete secret -n demo miniflux-internal-tls
$ kubectl get certificate -n demo miniflux-internal -o jsonpath='{.status.revision}'
2
$ kubectl get certificaterequest -n demo | grep miniflux-internal
miniflux-internal-2    True    True    openbao-cluster-internal
```

Neue Seriennummer, neuer Schlüssel (`rotationPolicy: Always`), neues Secret. Ein Pod, der das Secret als Datei mountet, sieht die neue Datei nach kurzer Zeit; einer, der es beim Start in den Speicher lädt, braucht einen Neustart – N° 4 beschreibt Reloader dafür.

Widerruf

Das alte Zertifikat ist noch bis Oktober gültig. cert-manager weiß das nicht mehr; OpenBao schon:

```
/ $ bao list pki_int/certs
3 Einträge

/ $ bao write pki_int/revoke serial_number=2e:5b:a9:60:...
revocation_time_rfc3339    2026-09-17T11:17:35Z
state                      revoked

/ $ bao read -field=certificate pki_int/cert/crl | openssl crl -noout -text | grep -A2 'Revoked'
Revoked Certificates:
  Serial Number: 2E5BA9605CB696BD585DA9E766D0AF337302027B
  Revocation Date: Sep 17 11:17:35 2026 GMT
```

Der Widerruf steht auf der CRL, die unter der URL aus `config/urls` abrufbar ist. Ob ein Client sie prüft, ist seine Sache – die meisten internen Dienste tun es nicht. Der Wert des Widerrufs liegt darum weniger in der technischen Wirkung als in der **Nachvollziehbarkeit**: OpenBao kennt jedes ausgestellte Zertifikat, seine Seriennummer und seinen Status. Mit aktiviertem Audit-Device (N° 1, Teil VI) auch, wer es beantragt hat.

Bei kurzen Laufzeiten – 30 Tage hier – ist der Ablauf ohnehin die wirksamere Sperre. Das ist der Grund, die `ttl` der Rolle klein zu halten.

Teil VII – Betrieb

Das Root verteilen

Damit Pods der neuen CA vertrauen, brauchen sie das Root-Zertifikat – nur das Root, nicht den Intermediate, der kommt mit jeder Kette mit:

```
$ bao read -field=certificate pki/cert/ca > root.pem
$ kubectl create configmap -n demo homelab-root-ca --from-file=ca.crt=root.pem
```

Pro Namespace eine ConfigMap ist für zwei Namespaces in Ordnung. Darüber hinaus ist **trust-manager** (vom cert-manager-Projekt) das Werkzeug: ein Bundle-Objekt, das eine CA in jeden Namespace synchronisiert. Es wurde hier nicht installiert, weil der Cluster es noch nicht braucht.

Die alte CA ablösen

homelab-ca stellt weiter Zertifikate aus. Der Umzug pro Dienst: issuerRef im Certificate (oder die Ingress-Annotation) auf openbao-cluster-internal ändern, privateKey.algorithm: ECDSA ergänzen, Secret löschen, warten. Der Dienst hat dann ein Zertifikat der neuen CA; seine Clients müssen das neue Root kennen. In der Übergangszeit vertrauen Clients beiden Roots – das ist der Grund, warum ein Root-Wechsel nie an einem Tag passiert.

Wenn kein Dienst mehr an homelab-ca hängt: ClusterIssuer löschen, Secret homelab-ca-key-pair löschen. Ab dann gibt es im Cluster keinen CA-Schlüssel mehr in etcd.

Root-Rotation, in fünf Jahren

Der Intermediate läuft 2031 ab, das Root 2036. OpenBaos Issuer-Modell ist dafür gebaut: Ein neuer Intermediate wird als zweiter Issuer im selben Mount angelegt, die Rolle zeigt per issuer_ref auf den neuen, der alte bleibt für die Kettenbildung bestehender Zertifikate erhalten, bis das letzte abgelaufen ist. Deshalb die Namen (int-2026) statt UUIDs.

Was in OpenTofu gehört

Alles aus openbao/pki-setup.sh – mit einer Einschränkung: generate/internal für das Root ist ein einmaliger Akt. vault_pki_secret_backend_root_cert kann ihn abbilden, aber ein versehentliches tofu destroy würde das Root löschen. Mounts, Rollen, Policies und die Auth-Rolle gehören in Tofu; die CA-Erzeugung selbst gehört in ein Runbook mit prevent_destroy.

Was noch offen ist

- Kein Audit-Device (N° 1, Teil V). Bis es aktiviert ist, weiß OpenBao was ausgestellt wurde, aber nicht wer es beantragt hat.

- Nur Server-Zertifikate. mTLS zwischen Diensten braucht eine zweite Rolle mit `client_flag=true` und einen Weg, Client-Zertifikate zu verteilen.
- Kein OCSP. OpenBao kann es (`ocsp_servers` in `config/urls`), Traefik fragt es nicht ab.

Anhang A – Alle Kommandos

```
# — OpenBao (siehe openbao/pki-setup.sh) —————
bao secrets enable -path=pki -max-lease-ttl=87600h pki
bao write pki/root/generate/internal common_name="Homelab Root CA" issuer_name=root-2026 key_type=ec key_bits=256 ttl=87600h
bao write pki/config/urls issuing_certificates=.../v1/pki/ca crl_distribution_points=.../v1/pki/crl
bao secrets enable -path=pki_int -max-lease-ttl=43800h pki
bao write -field=csr pki_int/intermediate/generate/internal common_name="Homelab Intermediate CA" key_type=ec key_bits=256 > pki_int.csr
bao write -field=certificate pki/root/sign-intermediate csr=@pki_int.csr format=pem_bundle ttl=43800h issuer_ref=root-2026 > pki_int.pem
bao write pki_int/intermediate/set-signed certificate=@pki_int.pem
bao write pki_int/issuer/<id> issuer_name=int-2026
bao write pki_int/config/urls issuing_certificates=.../v1/pki_int/ca crl_distribution_points=.../v1/pki_int/crl
bao write pki_int/roles/cluster-internal allowed_domains=... allow_subdomains=true key_type=ec ttl=720h max_ttl=2160h ...
bao policy write cert-manager - <<'EOF'
path "pki_int/sign/cluster-internal" { capabilities = ["create", "update"] }
EOF
bao write auth/kubernetes/role/cert-manager bound_service_account_names=cert-manager-openbao \
    bound_service_account_namespaces=cert-manager token_policies=cert-manager token_ttl=10m

# — cert-manager —————
kubectl apply -f k8s/clusterissuer.yaml
kubectl apply -f k8s/miniflux-certificate.yaml
kubectl apply -f k8s/miniflux-service-ingress.yaml
kubectl get clusterissuer,certificate,certificaterequest -A

# — prüfen —————
bao read -field=certificate pki/cert/ca > root.pem
openssl verify -CAfile root.pem -untrusted intermediate.pem leaf.pem
openssl s_client -connect <worker>:443 -servername miniflux.demo.example.internal -CAfile root.pem

# — erneuern / widerrufen —————
kubectl delete secret -n demo miniflux-internal-tls # erzwingt Neuausstellung
bao list pki_int/certs
bao write pki_int/revoke serial_number=<serial>
bao read -field=certificate pki_int/cert/crl | openssl crl -noout -text
```

Anhang B – Glossar

Root-CA / Intermediate-CA – Das Root signiert nur den Intermediate; der Intermediate signiert alles andere. Kompromittierung des Intermediate kostet einen neuen Intermediate, nicht ein neues Root.

CSR – Certificate Signing Request: öffentlicher Schlüssel plus gewünschte Namen, vom Besitzer des privaten Schlüssels signiert. Das Einzige, was cert-manager an OpenBao schickt.

Issuer (OpenBao) – Ein CA-Zertifikat samt (optionalem) Schlüssel in einem PKI-Mount. Ein Mount kann mehrere haben; `issuer_ref` wählt.

Rolle (PKI) – Regeln für Zertifikate: Domains, Schlüsseltyp, Laufzeit, Verwendungszweck. `sign/<rolle>` signiert einen CSR nach diesen Regeln.

sign vs. issue – `sign` nimmt einen CSR und gibt ein Zertifikat zurück; `issue` erzeugt auch den Schlüssel in OpenBao und gibt ihn heraus. `cert-manager` benutzt `sign`.

ClusterIssuer / Issuer – `cert-manager`-Objekt, das beschreibt, wo Zertifikate herkommen. Clusterweit bzw. pro Namespace.

ingress-shim – Der Teil von `cert-manager`, der aus `spec.tls` eines Ingress und der Annotation `cert-manager.io/cluster-issuer` ein Certificate erzeugt.

SAN – Subject Alternative Name. Die Namen, für die ein Zertifikat gilt; der CN ist nur noch Dekoration.

AIA / CRL-DP – URLs im Zertifikat, unter denen Clients das Aussteller-Zertifikat bzw. die Widerrufsliste finden.

rotationPolicy – Ob `cert-manager` bei der Erneuerung einen neuen privaten Schlüssel erzeugt (Always) oder den alten behält (Never).

Über den Autor

Thomas Zachmann ist freiberuflicher Platform Engineer in Hamburg. Er baut Enterprise-Plattformen für Kubernetes, Cloud und AI-Workloads – von Identity und Secrets über CI/CD und GitOps bis Observability – so, dass das interne Team sie danach ohne ihn betreiben kann. Diese Field Notes entstehen aus dieser Arbeit. Für Projektanfragen: thomaszachmann.de.

Der Schlüssel der CA liegt als Kubernetes-Secret in etcd. Jeder mit get secrets kann Zertifikate ausstellen. Niemand erfährt es.

Dieser Leitfaden verlegt die CA nach OpenBao: Root und Intermediate als getrennte Mounts, eine Rolle, die Domains, Schlüsseltyp und Laufzeit vorgibt, und cert-manager, der nur noch CSRs schickt. Ingress per Annotation, Service-zu-Service per Certificate, Kette gegen Traefik geprüft.

Mit zwei Ablehnungen, von denen die erste nicht die erwartete war: `role requires keys of type ec` kam vor `not allowed by this role`. Dazu Erneuerung mit neuem Schlüssel, Widerruf auf der CRL, und warum `set-signed` zwei Issuer erzeugt.

```
-- Das Einzige, was OpenBao je verlässt: ein CSR
bao write -field=csr pki_int/intermediate/generate/internal ...
bao write pki/root/sign-intermediate csr=@pki_int.csr ...
bao write pki_int/intermediate/set-signed certificate=@pki_int.pem
```



openbao	2.6.2 · pki/ + pki_int/
cert-manager	v1.20.2 · issuer vault
traefik	RKE2 chart 40.1 · SNI
keys	EC P-256 · 10y / 5y / 30d
policy	1 Zeile · sign only

21 Seiten · 7 Teile · 2 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.
CC BY 4.0 · Weitergabe erlaubt · ohne Gewähr